CER:HCC:TETHER: AI-supported Requirements Engineering

Tim Menzies (PI) and David Rieder (Co-PI) and Sarah Heckman (SP) NC State University

OVERVIEW

Generative AI now lets software engineering (SE) students build working systems faster than they can reason about who those systems serve. Requirements engineering (RE) instruction has not kept pace: it remains an up-front, document-centric exercise, finished before serious coding begins and never revisited. In time-pressured graduate courses, the result is technically fluent projects built on shallow, unexamined requirements.

We propose **TETHER**, an educational tool built on **Continuous Requirements Auditing (CRA)**. Where conventional checkers emit static scores ("Security: 2/5"), TETHER uses corpus-grounded analysis to audit a team's evolving artifacts (code, tests, documentation) for inconsistencies with the requirements, constraints, and regulations that the team itself declared. When implementation drifts from stated requirements, TETHER raises targeted, context-grounded questions at the moment of decision (e.g., "This login flow assumes always-on connectivity, yet your requirements identify rural users. How will you reconcile these?"). An on-device agent preserves privacy and responsiveness (goal: <5s on commodity student laptops, with an asynchronous fallback mode if that target proves infeasible). Our four-year plan evaluates CRA with over 600 students annually at NC State, via calibration against expert judgment, controlled experiments on feedback design, a cluster-randomized trial, and studies with Research Triangle industry partners. Deliverables: a validated and expert-calibrated CRA framework, empirical evidence on feedback design, an expert-annotated reference corpus of requirements, and open-source assessment instruments, all under open licenses.

INTELLECTUAL MERIT

This work targets a blind spot at the intersection of two literatures: RE pedagogy still treats requirements as a documentation phase divorced from implementation, while LLM-for-SE research concentrates on generating code rather than reasoning about requirements. CRA reframes RE as a continuous auditing practice: mechanically detecting requirement/implementation inconsistencies, then delivering the interpretive support students need to see why a mismatch matters and act on it in context. The technical claim is kept narrow and falsifiable: TETHER detects requirement/implementation inconsistencies, validated against expert-labeled ground truth, with learning outcomes measured by instruments independent of the tool. The feedback design is grounded in established results on attention direction (Norman), feedback framing (Bardzell), and the timing of behavioral triggers (Fogg), unified into a testable framework for AI-supported RE learning. A two-cohort pilot motivates the approach: in Fall 2024, nearly two-thirds of student projects were routine implementations of stock ideas; in Fall 2025, with prototype auditing support, the majority of projects instead showed substantive requirements work: conflict detection between requirements, accessibility analysis, and community-centered designs.

BROADER IMPACTS

As code generation is automated, the scarce skill becomes supervision: engineers who can hold AI-assisted development accountable to stakeholder needs and regulatory obligations. This project builds that capacity where it starts, in the SE classroom, and extends it to early-career professionals through industry studies. It also produces durable community infrastructure: validated instruments for measuring RE learning, evidence-based design principles for educational AI feedback, and open curricular materials, disseminated through SIGCSE TS and ICSE. More broadly, the project demonstrates a transferable pattern for educational AI. TETHER audits and questions student work, and that pattern applies in any domain where the goal is judgment as well as production.

KEYWORDS: Computing Education; Requirements Engineering; Large Language Models; Corpus-Grounded Feedback; Feedback Design; Human-AI Collaboration; Workforce Development.

CER:HCC:TETHER: AI-supported Requirements Engineering

Tim Menzies (PI) and David Rieder (Co-PI) and Sarah Heckman (SP) NC State University

1 Overview

Modern AI coding assistants have changed what a semester-long student project can be: implementations that once took weeks now appear in days. What has not changed is the hard part of software engineering (SE) that precedes and surrounds the code: requirements engineering (RE), the practice of establishing what a system must do, for whom, and under which business, societal, and regulatory obligations. Our graduate students arrive well-equipped to generate code and tests with AI, but with no comparable tooling for reasoning about requirements. The imbalance is consequential: the literature already describes RE as "at best an afterthought in many university software engineering curricula" [1], even though RE is exactly the discipline that equips developers to handle the socio-technical complexity of real systems. **We argue that weak RE preparation, amplified by AI-paced development, is now an urgent educational problem.** Because AI tools collapse the distance between an idea and running code, students routinely bypass the systematic analysis of stakeholder, business, and societal concerns that RE demands [1–4]. Left unaddressed, this produces graduates who can build impressive systems that answer to no one's actual needs.

Herein lies a paradox for graduate education: the faster students can build [5–7], the more (not less) requirements expertise they need. When implementation runs ahead of deliberation, projects exhibit what we call *requirements drift*: day-to-day coding decisions quietly override the stakeholder needs, ethical commitments, and regulatory constraints the team set out with [8]. Pedagogically, each drift episode is an opportunity in disguise: a concrete, teachable moment where a student can be shown that their code no longer meets what their requirements say. The challenge for computing educators is to catch those moments: to help students "retain the big picture" [9] amid the flood of implementation detail that AI tooling now generates.

Conventional RE instruction is poorly matched to this problem. It is organized around a single upfront deliverable: a requirements document, written and graded before serious coding starts, then rarely consulted again. Nothing in that structure gives students feedback while they are actually making design decisions, or connects requirements thinking to the code taking shape in front of them. Predictably, students come to treat RE assignments as paperwork standing between them and the "real" work of programming. Coverage gaps compound the problem: elicitation and modeling are taught, but safety, security, quality, and the ongoing management of requirements across the lifecycle are not [1, 10]. And the feedback machinery we do have (rubrics, lint-style warnings) demonstrably fails to change developer behavior [11]. What is needed instead is feedback that is *suasive*: engineered to provoke reflection [12], to arrive when it is timely and relevant [13, 14], and to help students build their own value frameworks rather than tick compliance boxes [15].

To address these pedagogical challenges, we propose TETHER, an AI-supported educational tool that serves as a personal *Rhetorical Coach* for graduate software engineering students. TETHER embeds requirements reasoning throughout the development cycle, helping students learn to:

- Integrate stakeholder documentation and regulations into problem analysis.
- Assess their projects against multiple value dimensions—from business viability (Table 1) to domain-specific maturity models for healthcare, cybersecurity, sustainability, and reliability (Table 2);
- Manage values-centered design arguments and develop rhetorical skills to justify and refine tech decisions;
- Experience continuous feedback loops that find and improve implicit requirements.

Prior work on AI in RE has established the technical feasibility of using AI to support requirements engineering tasks [3, 5, 6, 25]. However, there are gaps in current AI support for RE: reproducibility and consistency; hallucination risk; data quality and scarcity; tool maturity; integration of ethics, and lack of integration across the software development lifecycle [3, 6, 25]. Hallucination risk, in particular, is addressed head-on in this proposal: a human-in-the-loop calibration protocol (§3.2.4) targets Cohen's $\kappa > 0.6$ agreement with expert judgment, with per-domain validation and confidence-gated feedback that abstains

(deferring to the instructor) when retrieval coverage is thin, as can occur in niche, high-risk domains. Integration of AI into SE education provides the opportunity to enhance learning experiences and assessment while developing skills needed for professional practice [13]. However, there are concerns about student over-reliance, reduced critical thinking, and curriculum integration [13]. TETHER bridges these pedagogical challenges and technical gaps by grounding LLM analysis in a corpus of authoritative background documents (a lineage that begins with Retrieval-Augmented Generation [26]) and integrating it with Suasive Interface Theory [27]. Given student project artifacts, TETHER reviews these materials against background knowledge (legislation, safety standards, ethical frameworks). However, unlike passive linters or compliance checkers, TETHER uses LLMs to proactively engage students in requirements reasoning through timely prompts and interpretive cues that frame design decisions, highlight value tensions, and scaffold

Scale	**Business Oriented** Ref: [16] 10,000+ citations	**Social Impact** Ref: [17] 7,500 citations	**Human Flourishing** Ref: [18] 2,500+ citations	**Bloom's** taxonomy Ref1: [19] 85,000+ citations Ref2: [20] 45,000+ citations
level=1 (lowest)	One customer type, one value proposition.	**Individual.** One person at a time, no network effects.	**Narrow impact.** Convenience only, or effects just one domains.	**Remember/Understand.** Recalls or explains existing concepts.
level=2	**Multi-Sided.** Multiple customer segments, different value props.	**Community.** Neighborhood, campus, or small group benefits.	**Limited wellbeing.** Affects domains moderately.	**Apply (Basic).** Uses known techniques in standard ways.
level=3	**Platform (Early).** Enables limited 3rd-party participation, has APIs.	**Institutional.** Organization-wide or city-level systems.	**Moderate wellbeing.** Affects domains.	**Apply (Advanced).** Well-executed application, multiple integrations.
level=4	**Platform (Mature).** Robust ecosystem, 3rd parties build businesses.	**Societal.** Regional, industry-wide impact, potential policy influence.	**Substantial wellbeing.** Affects domains meaningfully.	**Analyze/Evaluate.** Breaks down problems, systems for evaluation.
level=5 (highest)	**Infrastructure.** Industry foundational layer, platforms built on it.	**Systemic.** International transformation, new infrastructure.	**Holistic flourishing.** Transforms multiple life dimensions.	**Create.** Produces genuinely novel solutions or combinations.
Might be used by...	Entrepreneurs, product managers, platform strategists.	Social entrepreneurs, policy makers, community organizers.	Ethicists, HCI researchers, human-centered designers.	Educators, curriculum designers, technical hiring managers.

Table 1: Domain General Evaluation Dimensions for Software Projects. Using LLMs, it is possible to summarize and score a wide range of software project artifacts on these scales. Also, LLMs can propose minimal changes that most improve a project (where "improve" means "drive it higher up these scales").

Scale	**Medical Records** (HIMSS EMRAM) Ref: [21]	**Cybersecurity** (OWASP SAMM) Ref: [22] Industry Standard	**Green Software** (GSMM) Ref: [23] Linux Foundation	**Reliability** (Well-Architected) Ref: [24] Cloud Standard
level=1 (lowest)	**Ancillaries Only.** Lab, Pharmacy, and Radiology systems not integrated.	**Ad-hoc.** Reactive security, developers perform basic checks only.	**Unaware.** No tracking of emissions; efficiency is accidental.	**Reactive.** Failures are handled manually after they occur.
level=2	**CDR Implemented.** Clinical Data Repository exists; might have conflict checking.	**Structured.** Repeatable security activities; basic checklists used.	**Aspiring.** Ad-hoc measurement of some emissions (e.g., server count).	**Defined.** Basic redundancy (backups) and recovery procedures exist.
level=3	**Documentation.** Nursing/clinical vitals and flow sheets are electronic.	**Consistent.** Comprehensive security auditing and architecture review.	**Acting.** Tracking Scope 1, 2, & 3 emissions systematically.	**Managed.** Automated health monitoring and alerting systems.
level=4	**CPOE & CDSS.** Computerized orders with clinical decision support active.	**Measured.** Metrics-driven security improvements; defects tracked.	**Optimizing.** Software logic supports demand shifting/shaping.	**Predictive.** Automated failover; recovery tested via game days.
level=5 (highest)	**Closed Loop.** Full EMR; analytics for population health & governance.	**Optimized.** Security fully automated in CI/CD; feedback loops active.	**Carbon-Aware.** Real-time adaptation to grid carbon intensity.	**Chaos-Ready.** Fault injection; system self-heals (Anti-fragile).
Might be used by...	Health informatics, hospital administrators, compliance officers.	CISOs, DevSecOps engineers, penetration testers.	Sustainability officers, cloud architects, Green IT leads.	Site Reliability Engineers (SREs), DevOps, Platform teams.

Table 2: Domain Specific Evaluation Dimensions for Software Projects. While Table 1 was very broad, this table shows domain specific maturity models (Healthcare, Security, Sustainability, Reliability).

progressive refinement of students' own ethical criteria. TETHER supports RE processes throughout the software development lifecycle.

Note that we use "retrieval" here in a mechanism-agnostic sense. Since Lewis et al. [26], grounding has become commodity infrastructure. Modern agentic coding tools can be pointed at a directory of documents and asked to summarize or cross-check them. Hence we make no novelty claim for retrieval itself: embedding-based search, agentic search over the corpus, and long-context ingestion are interchangeable back-ends in TETHER, and the choice among them is an empirical question inside the RQ1 ablations. Two things remain hard, and they are what this proposal contributes. Firstly, provenance: every intervention cites the specific requirement, standard, or regulation that triggered it. That citation trail is what makes confidence-gating and abstention possible (§3.2.4), and what lets students and instructors audit the feedback. Secondly, feasibility on commodity hardware: a 10,000-page corpus cannot be ingested wholesale into the context of a quantized model running locally on a 16GB student laptop. Hence, selective retrieval remains a hard requirement of the privacy-preserving local-first agent (§3.2.2), no matter how large frontier-model context windows grow.

TETHER employs **Continuous Requirements Auditing** (CRA) to transform static requirements assessment into a continuous feedback loop, monitoring development artifacts to detect drift from goals and prompting students to reflect and course-correct in real time. In traditional RE pedagogy, alignment is assessed once: students submit requirements documents, receive feedback, then proceed to implementation with little opportunity to revisit or refine their requirements thinking. CRA transforms this static checkpoint into a continuous learning process. As students develop their projects, CRA monitors emerging artifacts (design documents, code commits, test cases) and compares them against the intents, values, and constraints students have articulated during requirements analysis. When drift is detected, a gap between what is planned and the actual implementation, TETHER surfaces this misalignment through rhetorically framed interventions designed to prompt reflection rather than merely flag errors. This loop keeps requirements visible throughout development, so students come to see RE as a constant practice. It also delivers scaffolding at the moments when students are making consequential design decisions, and it gives students repeated chances to articulate, examine, and refine their value commitments. By operationalizing intent verification as a continuous rather than episodic activity, CRA helps students internalize the metacognitive habits that characterize expert requirements engineers: the capacity to notice when implementation choices diverge from stakeholder needs and to course-correct before drift compounds.

Critically, TETHER is non-prescriptive. Criteria like Table 1 and Table 2 are not "hard-wired" into TETHER. Instead, these criteria are artifacts created early by the instructor or early in the team's lifecycle, *by the teams themselves* to reflect the goals and values of the team and their community. Also, through iterative interaction with rhetorically designed interventions, TETHER helps students progressively articulate and refine their requirements value dimensions by learning to ask "What values does this design embody?" and "Whose needs might we be overlooking?" This pedagogical stance reflects our understanding that RE expertise involves not just applying checklists but developing the judgment to identify which stakeholders matter, which regulations apply, and which values should guide design in context-specific ways.

This work addresses three interconnected challenges in graduate computing education. First, how can we teach complex socio-technical reasoning in an era when AI tools enable rapid implementation? Students need educational support that matches the pace and style of modern development workflows. Second, how can we design AI-based educational tools that support learning rather than automating it away? TETHER must scaffold requirements reasoning without becoming a "requirements generator" that students use without understanding. Third, how can we scale personalized feedback in graduate courses where requirements complexity, spanning technical, legal, ethical, and social dimensions, exceeds what instructors can manually assess? TETHER must support just-in-time interventions to support student decisions, while providing an audit trail for instructors and researchers.

To assess how tools like TETHER enhance graduate RE education, we ask three research questions:

- **RQ1 (Technical Feasibility)**: How accurately can large language models detect inconsistencies between students' stated requirements and their implementation artifacts (code, tests, documentation), and what artifact combinations yield reliable inconsistency detection for educational feedback?

- **RQ2 (Pedagogical Mechanisms)**: How do interface-level framing cues and timely triggers influence student engagement with requirements reasoning, and what intervention designs most effectively support RE learning?
- **RQ3 (Educational Impact & Adoption)**: Does TETHER improve requirements engineering learning outcomes in authentic educational settings, and what conditions support effective adoption across diverse contexts?

TETHER is a specifically focused tool: it detects inconsistencies between stated requirements and implementation artifacts, is calibrated against expert judgment ($\kappa > 0.6$, §3.2.4), and is evaluated with learning measures independent of its own scoring (§4.4). Within that scope, TETHER is a coach. It helps students build the expertise required for responsible software development in an AI-accelerated world, and the judgment it coaches stays with the human.

2 Preliminary Results

To motivate this work, we offer preliminary results where corpus-grounded ideation exploration significantly altered and improved graduate student software projects.

A persistent challenge in graduate software engineering education is helping students move beyond conventional, technically-focused projects toward designs that demonstrate deeper engagement with stakeholder needs, societal values, and strategic differentiation. Longitudinal data from a decade of teaching graduate SE at NC State (2015–2026) revealed a troubling pattern: despite explicit instruction in requirements engineering principles, student projects consistently gravitated toward familiar, low-innovation themes (e.g., generic "exercise trackers," "menu assistants," and "calorie counters") that demonstrated technical competence but little evidence of nuanced requirements exploration. This pattern reflects a broader educational challenge: students struggle to translate RE concepts learned in lectures into meaningful design practices, defaulting instead to familiar solution patterns that minimize requirements complexity.

We hypothesized that this innovation blindspot stems not from lack of motivation but from lack of scaffolding. Without tools that support continuous requirements reasoning, students face a cognitive burden: they must simultaneously learn implementation skills, follow software engineering processes, manage team coordination, and engage in the unfamiliar practice of stakeholder analysis, often abandoning the latter when deadlines loom. **Continuous Requirements Auditing (CRA)**, operationalized through TETHER, addresses this challenge by embedding requirements reasoning into the workflow, providing timely feedback that keeps stakeholder values visible even as implementation pressures intensify.

To test whether CRA-supported tools could shift student project outcomes, we conducted an exploratory study comparing student projects (3-4 students per project) from Fall 2024 (Control, n=25 projects) and Fall 2025 (Experimental, n=35 projects):

- The Fall 2024 cohort received traditional RE instruction without TETHER support.
- The Fall 2025 cohort used a prototype version of TETHER, lacking the full suasive interface but possessing the core CRA scoring engine, to support an LLM-assisted ideation and requirements exploration.

2.1 Experimental Context and Methodology

The educational intervention centered on scaffolding students' engagement with stakeholders, documentation, and values, which are areas where traditional RE pedagogy most often fails to translate into practice.

Stakeholder and Documentation Exploration. Student teams were given ten use cases for a food ordering system. Rather than proceeding directly to implementation (the typical student response), teams used TETHER's retrieval workflow [26] to query a domain-specific corpus comprising approximately 10,000 pages of legal, medical, and regulatory texts related to e-commerce platforms and health and safety. This scaffolded exploration served explicit learning objectives: students practiced identifying relevant external constraints, discovered stakeholders absent from the initial problem statement (e.g., delivery workers, dietary-restricted users, local restaurants), and encountered regulatory frameworks they would not have considered independently. Critically, students were guided to ask generative questions: "What require-

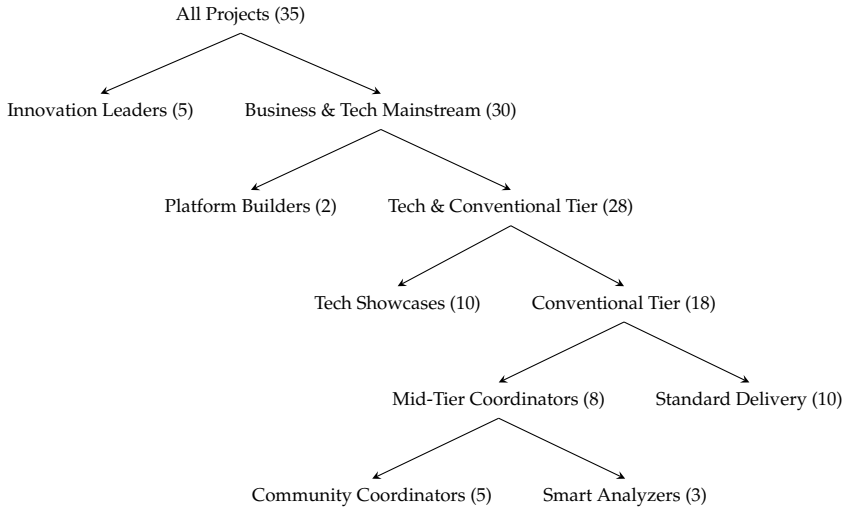


Figure 1: Hierarchy of student project clusters (2025 data) report by TETHER's LLMs (data from Table 4 based on 35 student projects).

Cluster	Definition & General Impact
Innovation Leaders	High social impact with strong wellbeing benefits across multiple life dimensions. Balance technical execution with genuine societal value through health, social connections, and community benefit.
Platform Builders	Infrastructure-focused projects enabling ecosystems through APIs and extensibility. Demonstrate platform economics and architectural openness for third-party value creation.
Tech Showcases	Sophisticated AI/LLM applications with high cognitive complexity. Excel in novel algorithms but focus on individual benefit without broader social impact.
Community Coordinators	Emphasize local social connection within neighborhoods or campus. Facilitate group interactions, shared resources, and collective benefit at human scale.
Smart Analyzers	Mid-tier intelligence with recommendation engines or optimization algorithms. Apply analytical thinking for individual decision-making without systemic reach.
Standard Delivery	Conventional food delivery or marketplace implementations. Demonstrate technical competence but lack differentiation in impact or business model.

Table 3: Student Project Cluster Definitions and General Impact

ments are missing?" and "What changes could produce a more distinctive product?", questions that model expert RE reasoning but that novices rarely ask spontaneously.

Values-Centered Assessment. The TETHER prototype utilized local LLMs (via the Ollama environment) to assess student artifacts against the multi-dimensional value dimensions framework shown in Table 1. The automated assessment pipeline operationalized CRA through three steps:

1. **Dimension Ingestion:** Loading criteria definitions spanning Bloom's cognitive complexity, human flourishing, social impact, and business model sophistication (Table 1).
2. **Artifact Review:** Ingesting student project documentation to establish baseline intent.
3. **Scoring and Feedback:** Prompting LLMs to score artifacts against value dimensions and generate minimal interventions: concrete suggestions for improving alignment with stated values.

This process made values reasoning concrete and discussable. Rather than abstract exhortations to "consider stakeholders," students received specific feedback: "Your current design serves individual users (Social Impact: Level 1). Adding group ordering features would extend benefits to communities (Level 2) and align with the collaborative dining patterns identified in your stakeholder analysis."

2.2 Results: Shifting Student Project Outcomes

To characterize changes in student project quality, we applied Greedy Agglomerative Clustering to TETHER-generated scores, yielding the hierarchical taxonomy in Figure 2. Table 3 provides semantic defi-

	%						
	2024 (n=25)	2025 (n=35)	Bloom's	Flourish	Impact	Business	Profile
Innovation Leaders	4	14.3	3.8	4.0	4.0	2.0	High social impact with strong wellbeing benefits
Platform Builders	0	5.7	4.5	2.0	2.5	4.0	Ecosystem thinking, infrastructure focus
Tech Showcases	28	28.6	5.0	2.4	2.4	2.3	High AI/LLM innovation, individual benefit focus
Community Coordinators	4	14.3	3.6	2.4	2.6	2.0	Local social connection emphasis
Smart Analyzers	32	8.6	4.0	1.7	2.0	2.0	Mid-tier intelligence with tracking features
Standard Delivery	32	28.6	3.0	1.8	1.9	1.9	Conventional food delivery implementations

Table 4: Clusters found in student projects, using the criteria of Table 1 (comparing 2024 (n=25) and 2025 (n=35) cohorts). For a visualization of columns 1,2,3 please see Figure 2.

nitions of each cluster, while Table 4 and Figure 3 show quantitative distributions across cohorts.

The central finding is that CRA-supported ideation shifts the distribution of student projects to higher innovation and deeper values engagement. As seen in Figure 2, in 2024, without TETHER support, two-thirds of projects (64%) fell into the bottom two clusters, "Standard Delivery" and "Smart Analyzers", characterized by conventional implementations with limited stakeholder consideration. In 2025, with TETHER support, this pattern reversed: most projects (65%) moved to the upper clusters, demonstrating more sophisticated engagement with social impact, human flourishing, and platform thinking.

Most striking was the emergence of an "Innovation Leaders" cluster (14.3% of 2025 projects) that was nearly absent in 2024 (4%). These projects demonstrated not just technical sophistication but genuine engagement with the RE practices TETHER scaffolded. Two examples illustrate this qualitative shift:

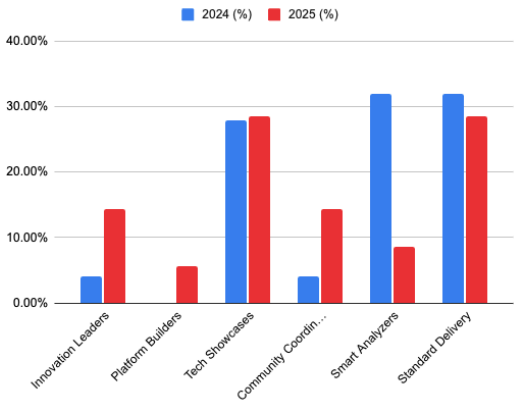


Figure 2: 2024:2025 = without:with TETHER.

- **ReqTrace** (Figure 4.a) uses natural language processing to automatically generate knowledge graphs for conflict detection in requirements specifications. This project emerged directly from students' TETHER-supported exploration of how large requirements documents create coordination challenges, a stakeholder pain point they discovered through guided corpus exploration.
- **Clarity-AI** (Figure 4.b) uses LLMs and NLP to convert requirements edge cases to candidate test cases for user review. Students developed this concept after TETHER highlighted that their initial requirements lacked testability, a values dimension (verification rigor) they had not previously considered.

2.3 Evidence of Learning: From Scoring to Improvement

Beyond shifting aggregate outcomes, TETHER demonstrated the capacity to support individual student learning through concrete, targeted feedback. To illustrate, we analyzed "App0," a project initially classified in the lowest-performing cluster (Total Score: 7). TETHER's CRA engine identified specific misalignments between the team's stated intentions (serving diverse community needs) and their actual design (a conventional single-user delivery app). The system proposed targeted architectural transformations:

1. Integrating philanthropic APIs to enable meal donation features (addressing the team's stated but unimplemented interest in food insecurity);
2. Implementing group booking features to enhance social impact (moving from Individual to Community on the social impact scale);
3. Exposing APIs for local business integration (going from a closed system toward platform thinking).

We estimate that those changes would elevate App0 from "Standard Delivery" to the "Community Coordinators" cluster (Projected Score: 12). Note the difference from grading. A grade tells a team where it fell

Figure 3.a: *ReqTrace: find requirement conflicts.*

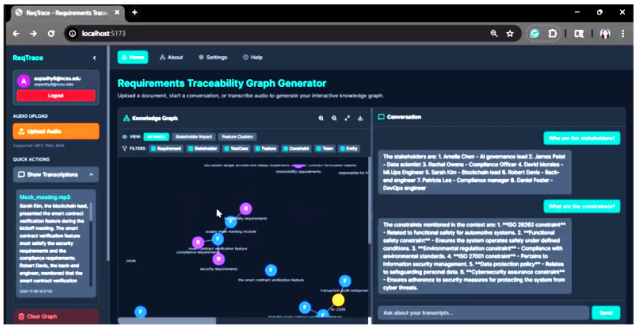


Figure 3.b: *Clarity-AI: build tests from requirements.*

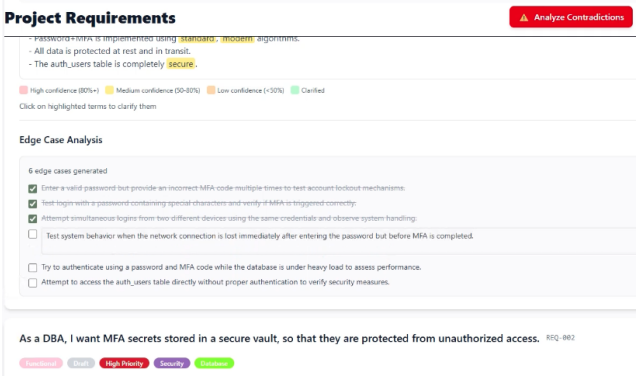


Figure 3: Two "innovation leader" projects (as defined in Table 3) from 2025.

short; this feedback listed specific, achievable changes that would better align the implementation with the team's own stated values.

2.4 Portfolio-Level Insights: Informing Curriculum Design

TETHER's CRA capabilities extend beyond individual feedback to portfolio-level analytics that can inform curriculum design. Aggregated analysis of the 2025 cohort revealed systemic patterns:

- **71% of projects** showed low "Flourishing" scores, suggesting that human wellbeing considerations remain under-emphasized, even with TETHER support;
- **83% of projects** lacked platform extensibility, indicating that ecosystem thinking requires more explicit scaffolding than our prototype provided.

These diagnostics suggest course- and curriculum-level interventions. For example, TETHER recommended mandating a "Flourishing Impact Statement" in project proposals, a lightweight assignment that would prompt early consideration of wellbeing dimensions. Simulations suggest this policy would increase the cohort's mean Flourishing score by 47%.

Hence TETHER serves instructors as well as individual students. Portfolio analytics show where a pedagogy succeeds and where it needs more scaffolding, and they give computing education researchers a new instrument for studying how specific interventions affect learning across a cohort.

2.5 Conclusion from Preliminary Study

The above suggests that current LLMs can detect strategic misalignments in graduate software projects, and that CRA-style feedback is associated with a shift in student work toward deeper stakeholder and values engagement. §2.6 lists threats to the validity of that conclusion; §4 describes how the proposed work resolves each one.

2.6 Threats to Validity of these Preliminary Results

These results motivate, but do not prove, CRA's educational value. The study was quasi-experimental: the 2024 and 2025 cohorts may have differed in assignment framing, instructional materials, and (importantly) ambient familiarity with LLM tools, which grew rapidly between those semesters. The intervention was also bundled: the 2025 cohort received both LLM-assisted ideation and CRA scoring, so the observed shift cannot be attributed to CRA alone. Finally, the outcome measure was TETHER's own scoring, so improvements could partially reflect alignment with the model's internal criteria rather than genuine gains in requirements reasoning. The proposed work is designed around these threats: a cluster-randomized trial with outcome measures independent of TETHER (§4.4), an ablation arm comparing TETHER with and without continuous verification to isolate the CRA mechanism, and expert blind review of project samples.

3 Proposed Extensions: Operationalizing CRA for RE Education

The prototype used in our preliminary study was limited in ways that constrain both pedagogical effectiveness and our ability to rigorously evaluate learning outcomes. Students received feedback only after submitting artifacts, missing the continuous scaffolding that could support learning during development. The corpus of background documents was instructor-curated, denying students the opportunity to practice the critical RE skill of identifying relevant constraints. And we lacked the instrumentation needed to understand *how* students engaged with feedback (whether they reflected on suggestions, discussed them with teammates, or simply ignored them). Hence the four technical extensions below, which turn TETHER from a post-hoc assessment tool into a continuous learning environment for requirements engineering. Each extension addresses specific educational challenges and directly supports the investigation of our research questions. Table 5 maps these extensions to our four-year timeline and associated RQs.

3.1 Pedagogical and Theoretical Foundations

Correct information, delivered once, does not change how students think about requirements. Hence we ground TETHER's pedagogical design in three complementary theoretical frameworks about attention, framing, and timing.

Directing Attention to Requirements Reasoning. Norman's *Theory of Signifiers* [28] demonstrates that interface elements direct user attention toward certain aspects of a system while making others less salient. Students naturally attend to implementation details (e.g., compile errors, test failures) because development environments make these salient. Requirements considerations, like stakeholder needs, value alignments, and regulatory constraints, remain invisible unless deliberately surfaced. TETHER applies signifier theory by designing interface elements that make requirements reasoning as visible and immediate as syntax errors.

Framing Requirements as Arguments. Bardzell's *Interaction Criticism Framework* [29, 30] shows that interfaces operate rhetorically: they construct arguments about what matters and how users should respond. For RE education, this means that *how* TETHER presents feedback shapes *what* students learn. A warning that says "Security score: Low" teaches students that security is a checkbox. A framing that says "Your authentication design assumes users have reliable internet access. How might this affect users in rural areas served by your food delivery system?" teaches students that security connects to stakeholder contexts. TETHER treats every intervention as a rhetorical move designed to model expert RE reasoning.

Timing Interventions for Learning. Fogg's *Behavior Model* [31] emphasizes *kairos*, the opportune moment to persuade. Students are most receptive to requirements feedback when actively making design decisions, not when reviewing grades days later. TETHER operationalizes kairos by delivering CRA feedback during development, when students have both the motivation (they want their project to succeed) and the ability (they can immediately act on suggestions) to engage with requirements reasoning.

Continuous Requirements Auditing as Pedagogical Mechanism. CRA integrates these theoretical foundations into a coherent pedagogical approach. By continuously monitoring student artifacts against declared intents and values, CRA creates ongoing opportunities for attention-direction (Norman) [28], interpretive framing (Bardzell) [29, 30], and timely triggering (Fogg) [31]. When CRA detects drift, TETHER surfaces this misalignment using rhetorically framed interventions designed to prompt reflection and support immediate correction. The resulting loop is light enough not to disrupt development flow. That said, it keeps requirements visible across the whole project lifecycle, and it generates the interaction logs that our educational research needs.

3.2 Technical Extensions for Educational Impact

To address the limitations of our prototype and enable rigorous investigation of our research questions, we propose four technical extensions. Each addresses specific educational challenges while creating the instrumentation needed for computing education research.

3.2.1 Extension 1: Dynamic Knowledge Base Construction

Educational Challenge: In our prototype, students queried an instructor-curated corpus of 10,000 pages. While this scaffolded their exploration, it constrained a critical learning opportunity. Expert require-

ments engineers do not work from pre-assembled document sets. They must identify which regulations, standards, competitor analyses, and stakeholder perspectives are relevant to their specific context. This knowledge curation skill is central to RE expertise but absent from traditional RE pedagogy, which typically provides students with pre-specified requirements rather than asking them to discover constraints.

Proposed Solution: We will extend TETHER to support **User-Driven Corpus Ingestion**, allowing student teams to scrape, curate, and index their own domain-specific documents. Students might gather current FDA guidelines for a health-related application, municipal regulations for a civic technology project, or accessibility standards for an educational tool. TETHER will scaffold this process by suggesting document types to consider, flagging potential gaps in coverage, and helping students assess source credibility.

Connection to RQ1: This extension directly supports investigation of RQ1. By varying the composition and quality of student-curated corpora, we can conduct ablation studies that reveal which document types contribute most to CRA accuracy. If student-curated corpora yield feedback comparable to expert-curated corpora, this validates the approach; if quality degrades significantly, this identifies where additional scaffolding is needed. Students develop skills in identifying relevant external constraints, evaluating source credibility, and understanding how background knowledge shapes requirements reasoning.

3.2.2 Extension 2: The Local-First Developer Agent

Educational Challenge: Our prototype assessed student projects after artifact submission, timing that mirrors traditional grading but misses the pedagogical opportunity of real-time feedback. Learning research consistently shows that immediate feedback supports skill acquisition more effectively than delayed feedback [32]. Moreover, when students receive RE feedback only after completing work, they experience requirements as evaluation criteria rather than design guidance. To help students internalize RE as a continuous practice, TETHER must provide feedback *during* development.

Proposed Solution: We will develop a **Local-First Developer Agent**: a CLI tool and IDE plugin that students can invoke on demand throughout development. The agent operates primarily via local LLMs (e.g., quantized Llama models running on student hardware). This design ensures privacy (student code never leaves their machine), eliminates API costs that could create access inequities, and enables the low-latency responses (stretch target: <5 seconds; fallback: asynchronous "audit mode" feedback, see §3.4) necessary to maintain developer flow. A "suasive intervention" that interrupts a student's train of thought fails. One that arrives while the student is contemplating a design decision succeeds.

Connection to RQ2: This extension enables investigation of RQ2. The Local-First Agent creates infrastructure for A/B testing of intervention designs: comparing rhetorically framed triggers ("Your payment processing doesn't address the privacy concerns raised in your stakeholder analysis. How might you reconcile this?") against static warnings ("Privacy score: Low") to measure differential effects on engagement and learning. We can also investigate latency effects: does feedback delivered in 2 seconds produce different learning outcomes than feedback delivered in 10 seconds? Students will experience RE as an ongoing practice integrated with development, developing the metacognitive habit of considering requirements implications while coding.

3.2.3 Extension 3: Portfolio-Level Telemetry for Educational Intelligence

Educational Challenge: Individual feedback helps individual students, but instructors lack visibility into how their pedagogy affects learning across an entire cohort. Are students struggling with specific RE dimensions (e.g., security considerations)? Do certain project types consistently neglect stakeholder diversity? Are improvements in RE reasoning distributed across students from different educational backgrounds? Without portfolio-level data, instructors cannot identify systematic gaps in their curriculum or evaluate whether pedagogical interventions produce measurable improvements.

Proposed Solution: We will build a **Privacy-Preserving Telemetry Dashboard** that aggregates CRA data across student projects without transmitting sensitive code or documentation. Local agents transmit only dimension scores (e.g., "Project A: Security=2, Flourishing=4") and interaction metadata (e.g., "Team viewed suggestion; no action taken within 24 hours"), creating a "heatmap" of cohort-level RE engagement. The dashboard supports gap identification across RE dimensions, intervention tracking (measuring whether curriculum changes produce measurable shifts), monitoring of differential effectiveness across

student backgrounds (e.g., prior programming experience, educational pathway, institution type), and longitudinal analysis of RE capability development.

Connection to RQ3: This extension supports investigation of RQ3. Portfolio-level telemetry enables us to study not just whether TETHER improves individual projects but whether it produces systemic improvements in how cohorts engage with RE. We can investigate whether improvements persist after scaffolding removal and how organizational factors (team composition, project type, course structure) moderate effectiveness. Instructors benefit from responsive pedagogy: when 70% of teams neglect accessibility, they can provide targeted instruction before projects are completed and see any impact from the instruction.

3.2.4 Extension 4: Calibration and Ground Truth for Trustworthy Feedback

Educational Challenge: LLM-based feedback carries risks of hallucination and inconsistency that are particularly problematic in educational contexts. If TETHER provides inaccurate feedback (telling students their security design is adequate when it has critical flaws, or flagging non-issues that waste student time), it undermines both learning and trust. Students who experience unreliable feedback learn to ignore the tool. Without ground truth validation, we cannot make credible claims about educational effectiveness.

Proposed Solution: We will implement a **Human-in-the-Loop Calibration Protocol**: (1) **Reference Set Development**, where expert reviewers (course TAs, industry practitioners) assess a representative sample of student projects, creating validated comparison data; (2) **Disagreement Analysis**, which identifies systematic biases where the LLM consistently over- or under-estimates, or project types that confuse the model; (3) **Prompt Refinement**, which iteratively improves alignment with expert judgment while maintaining rhetorical framing; (4) **Confidence Calibration**, which surfaces confidence levels (e.g., "High confidence: grounded in 3 retrieved regulations" vs. "Low confidence: extrapolated from general principles") so students understand when to trust feedback and when to seek additional guidance. For niche or high-risk domains (e.g., safety-critical or cyber-physical software) where corpus coverage is thin and hallucination risk is highest, confidence gating is deliberately conservative: low-coverage retrievals trigger abstention and referral to the instructor, rather than speculative feedback.

Connection to RQ1 and RQ2: This extension is essential for both RQ1 (establishing that CRA inferences are accurate) and RQ2 (ensuring that framing cues align with expert judgment rather than LLM artifacts). We target Cohen's $\kappa > 0.6$ (moderate-to-strong agreement) between TETHER and human experts, sufficient for educational deployment. Calibration data will also reveal whether rhetorically framed feedback is more or less accurate than neutral feedback.

Learning Outcomes: Calibration ensures that students receive feedback they can trust, creating the foundation for productive engagement with CRA suggestions. Additionally, by making confidence levels visible, TETHER teaches students an important meta-skill: understanding when AI-generated guidance is reliable and when human judgment is needed, a capability increasingly important as AI tools proliferate in software development.

3.3 Responsible AI and Ethical Safeguards in Educational Contexts

TETHER's persuasive design raises concerns amplified by instructor-student power differentials. We address these through five principles:

- **Autonomy:** All interventions are advisory; students may dismiss suggestions without penalty. The system never blocks commits or mandates action.
- **Transparency:** Content is labeled "AI-generated," persuasive mechanisms are explained, and instructors explicitly teach students to critically evaluate TETHER's rhetorical framing.
- **Privacy:** Local-first architecture ensures student work never leaves their machines; only de-identified, aggregated scores reach the telemetry dashboard when explicitly enabled.
- **Mitigating Over-Reliance:** Confidence displays highlight uncertainty, low-confidence suggestions prompt independent verification, and "audit mode" provides post-hoc feedback for unscaffolded practice.
- **Broad Access:** Target hardware requirements (16GB RAM, no GPU) minimize access barriers, ensuring the tool works for all students everywhere regardless of personal computing resources; learning outcomes will be disaggregated by non-protected factors such as prior programming experience, educational pathway (traditional CS major, career changer, transfer student), and undergraduate institution type, to

identify where the tool serves some students less well than others; where differential effectiveness is identified we will respond, not merely report: targeted scaffolding revisions and, where differences persist, condition-level design changes reviewed quarterly with the IRB. Quarterly IRB reviews will monitor for student reactance, automation bias, and differential effectiveness, with findings informing iterative refinements.

3.4 Prioritization: Core versus Exploratory Components

Reviewers of ambitious systems proposals rightly ask which components are essential and which can be shed. We distinguish them explicitly. **Core** (must succeed for the project to deliver its central claims): the CRA scoring engine, the human-in-the-loop calibration protocol (§3.2.4), and the CSC 326 randomized trial (§4.4). **Exploratory** (high value, but the core results stand without them): dynamic corpus ingestion (§3.2.1), the telemetry dashboard (§3.2.3), and the industrial sprints. Each exploratory component has a fallback: if student-curated corpora underperform, we revert to instructor-curated corpora (as in the pilot); if the local-first agent cannot meet interactive latency on commodity hardware, we degrade to asynchronous "audit mode" feedback or opt-in cloud inference with equivalent privacy guarantees; if telemetry proves too complex, portfolio-level analysis proceeds via manual sampling. This ordering is reflected in the timeline (Table 5): core components occupy Years 1–3, and exploratory components are scheduled so that their failure does not block the trial.

4 Research Plan and Evaluation

This project investigates how AI-supported tools can enhance graduate software engineering education, specifically addressing the challenge of helping students develop requirements engineering (RE) expertise in an era of AI-accelerated development. Our research follows a deliberate progression:

- We first establish technical foundations for educational integration;
- Next, we investigate pedagogical mechanisms through controlled experimentation;
- Finally, we evaluate educational impact in authentic settings, with extended industrial validation.

Table 5 shows the timeline of this research plan, and Table 6 summarizes our evaluation measures and success criteria.

4.1 Research Context and Participants

Primary Educational Testbed. A significant asset of this proposal is guaranteed access to a large-scale, longitudinal educational testbed. PI Menzies and SP Heckman teach the primary software engineering courses in NC State's Computer Science program:

- **CSC 510 (Software Engineering):** Graduate-level course emphasizing professional SE practices, team-based development, and requirements engineering. Enrollment: $\tilde{1}50$ students annually. Students work in teams of 4-5 on semester-long projects with industry-relevant scope.
- **CSC 326 (Software Engineering Fundamentals):** Undergraduate course introducing SE principles to CS majors. Enrollment: $\tilde{5}00$ students annually. Students complete smaller-scale projects that provide controlled conditions for comparative studies.

Together, these courses provide access to over 600 students annually, offering the statistical power necessary for rigorous educational research. The graduate/undergraduate distinction lets us test how TETHER effectiveness varies with student experience level (a significant question for curriculum integration).

Participant Backgrounds. NC State's CS program enrolls students who arrive by many pathways: traditional CS majors, career changers, transfer students, and students from a wide range of undergraduate institutions with correspondingly varied technical preparation. Our analysis will disaggregate outcomes by these background factors (prior programming experience, educational pathway, and undergraduate institution type) to identify where TETHER is more or less effective and to ensure the tool creates opportunities for all students.

Ethical Considerations. Participation in research activities will be voluntary; students may opt out without affecting their course standing or access to TETHER. Course grades will be determined indepen-

	Year 1				Year 2				Year 3				Year 4			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
Phase 1: Technical Foundation (RQ1)																
TETHER v1.0 Core Engine **(RQ1a)**																
Dynamic Corpus Ingestion **(RQ1b)**																
Interface Design & Prototyping																
CSC 510 Pilot (n=15–20 teams) **(RQ1a,c)**																
Phase 2: Pedagogical Mechanisms (RQ2)																
Ground Truth Study **(RQ2a)**																
Local LLM Optimization **(RQ2c)**																
Rhetorical A/B Test **(RQ2b)**																
Telemetry Dashboard Build																
Phase 3: Educational Impact (RQ3)																
CSC 326 RCT (n≈500) **(RQ3a)**																
Industrial Sprint I (n=10–15) **(RQ3b)**																
Expert Panels—Initial **(RQ3a,b)**																
Industrial Sprint II (n=15–20) **(RQ3b)**																
Expert Panels—Extended **(RQ3a,b)**																
Longitudinal Tracking & Fading Study																
Final Handbook & Open Release																

CSC GRAs (1.5 FTE): Technical/Architecture; **English GRA (1.0 FTE): Pedagogy/Interface**; **Joint Effort**.

Table 5: Project Timeline mapped to Research Questions.

Table 6: Evaluation Framework: Mapping RQs to Methods and Success Criteria.

RQ/Phase	Methodology	Success Criteria
RQ1	Suggestion Acceptance Rate (SAR: % of suggestions acted on within 48h); TA agreement ratings	SAR >30%; TA agreement >70%; usefulness ≥3.5/5
(Year 1)	Ablation studies (25–100% corpus coverage)	<15% degradation at 50% coverage; curation ≥75% of expert
	Incomplete artifact analysis; fallback prompts	<20% loss at 50% completeness; >60% respond to prompts
RQ2	Ground truth study (experts n=10–14 vs. TETHER)	Fleiss' κ >0.5; Cohen's κ >0.6; no dimension >30% disagree
(Year 2)	A/B test: neutral vs. rhetorical framing (n≈150)	Rhetorical >1.5× engagement; rationale p <.05; transfer d >0.3
	Latency benchmarks (16GB, no GPU); timing logs	<5s for 90% requests; identify threshold & teachable moments
RQ3	Cluster-RCT in CSC 326 (n≈500); pre/post assessment	RE gains d >0.3; rationale quality p <.05
(Years 3–4)	Education Expert Panel (n=6–8, blind review)	>70% endorse improved RE reasoning
	Industrial sprints (n=10–15 Y3; n=15–20 Y4); thematic analysis	SAR comparable; >70% rate valuable; characterize facilitators
	Subgroup analysis; fading study; multi-semester tracking	No negative differential; practices persist; prior exposure predicts RE

dently of research participation. We will use wait-list control designs where feasible, ensuring all students receive TETHER support.

4.2 RQ1: Technical Foundation for Educational Feedback (Year 1)

How accurately can large language models detect inconsistencies between students' stated requirements and their implementation artifacts, and what artifact combinations yield reliable inconsistency detection for educational feedback?

Before TETHER can support RE learning, we must establish that CRA assessments are sufficiently accurate to serve as educational feedback. We deliberately anchor RQ1 to a measurable construct (detected inconsistency between stated requirements and implementation) rather than direct inference of "intents" or "values," which are not directly observable in student artifacts. Values language remains in TETHER's feedback vocabulary, but all accuracy claims are grounded in inconsistency detection scored against expert-

labeled ground truth (§3.2.4). We employ CSC 510 graduate students as co-designers and early evaluators, drawing on their maturity to provide substantive feedback. Table 5 shows our activities in this area. Table 6 summarizes all measures and success criteria.

Initially, we will develop TETHER v1.0 with core CRA scoring engine, dimension visualization dashboards, and Local-First Developer Agent (CLI and IDE plugins). We deploy with CSC 510 teams (n=15–20) throughout semester projects, tracking Suggestion Acceptance Rate (SAR), the percentage of suggestions resulting in observable student action within 48 hours, and conducting end-of-project interviews (10–12 teams) exploring how TETHER influenced requirements thinking.

Students produce diverse artifacts, but not all equally reveal requirements reasoning. Using archived CSC 510 projects, ablation studies will systematically vary corpus composition (25%, 50%, 75%, 100% coverage), comparing student-curated versus expert-curated corpora to identify artifact types that enable accurate assessment while constituting valuable learning activities. These ablations will also compare grounding back-ends (embedding-based retrieval, agentic search over the corpus, and long-context ingestion where model and hardware permit), so that TETHER's architecture tracks the state of practice rather than committing to any one retrieval mechanism. Because students rarely produce complete documentation, especially early when formative feedback is most valuable, we will develop fallback mechanisms, including prompts for intent clarification and confidence-weighted feedback that acknowledges uncertainty.

4.3 RQ2: Pedagogical Mechanisms for Effective Feedback (Year 2)

How do interface-level framing cues and timely triggers influence student engagement with requirements reasoning, and what intervention designs most effectively support RE learning?

In this stage, we will investigate how feedback *presentation* affects learning, drawing on Norman's signifier theory [28], Bardzell's rhetorical framing [29,30], and Fogg's behavior model [31]. We first establish alignment between AI and expert judgment through a Human-in-the-Loop Calibration Protocol: expert raters (TAs n=6–8, industry advisors n=4–6) independently assess 50 archived projects, calculating inter-rater reliability with Fleiss' κ for human baseline and Cohen's κ between TETHER and human consensus. Systematic disagreement analysis identifies dimensions where TETHER may be unreliable.

Within-subjects experiment with CSC 510 students (n≈150) compares neutral notifications (e.g., "Security: 2/5") against rhetorically framed interventions (e.g., "Your authentication design assumes reliable internet. How will this affect rural users you identified as underserved?"). Each team experiences all conditions across project phases (counterbalanced). Transfer tasks assess RE reasoning without TETHER.

Fogg's model emphasizes kairos, which is feedback during active decision-making [31]. Kairos-driven feedback produces different effects than delayed feedback. To maximize feedback in the moment, we optimize the Local-First Agent for student hardware (16GB RAM, no GPU) through model quantization and caching, with latency tolerance studies (1s–30s delays), identification of engagement thresholds, and characterization of high-receptivity phases in development workflows.

4.4 RQ3: Educational Impact and Ecological Validity (Year 3-4)

Does TETHER improve requirements engineering learning outcomes in authentic educational settings, and what conditions support effective adoption across diverse contexts?

In this final stage, spanning Years 3–4, we test whether technical validity and pedagogical mechanisms translate into genuine educational impact. Student project quality improvements do not necessarily indicate learning. Students might produce better artifacts by following suggestions without understanding why. A cluster-randomized controlled trial in CSC 326 (n≈500) assigns lab sections to TETHER or control conditions, with pre/post RE reasoning assessments measuring stakeholder identification, value reasoning, and constraint recognition. Critically, all primary outcome measures are independent of TETHER's own scoring: instrument-based assessment of stakeholder identification accuracy, requirements completeness, and traceability quality, plus expert blind review. TETHER-generated dimension scores serve only as secondary process measures. This avoids the circularity of evaluating TETHER-supported projects with TETHER itself. An Education Expert Panel (n=6–8, via SP Heckman's SIGCSE network) blind-reviews project samples to validate educational significance; all outcomes are disaggregated by prior programming

experience and educational pathway to identify differential effectiveness.

To assess professional transfer, we will partner with Research Triangle companies. NC State Computer Science manages an extensive and active alumni program for all its graduates working in the nearby Research Triangle Information Technologies industry (and also with computer companies working downtown in Raleigh, such as RedHat). Based on recent experience, we anticipate no problems in gathering industrial volunteers to aid in this work.

Using those industrial contacts, we will conduct studies in two phases: an initial industrial sprint in Year 3 with early-career developers (n=10–15), followed by an expanded sprint in Year 4 (n=15–20) incorporating lessons learned. These sample sizes are sufficient for the planned thematic analysis: qualitative saturation is typically reached at n≈12–15 interviews [**?**]. Semi-structured interviews with developers (n=15–20) and managers (n=6–10) explore facilitators, barriers, and organizational conditions; an Industrial SME Panel (n=6–8) evaluates utility. Longitudinal tracking in Year 4 compares RE practices of CSC 510 students with/without prior TETHER exposure in CSC 326, and a scaffolding-fading study progressively reduces feedback frequency to measure practice persistence.

4.5 Deliverables and Dissemination

Our work concludes with synthesis and open release: **TETHER Adoption Handbook** for educators; validated **assessment instruments** for RE learning; **design principles** for educational AI feedback; **replication package** with anonymized data and code; and **open-source TETHER release** with complete documentation. The "Reference Set" of expert-annotated requirements supports future research in human-AI collaboration for SE education.

5 Intellectual Merit

This proposal addresses a fundamental challenge: teaching requirements engineering when AI tools enable students to produce sophisticated implementations without commensurate growth in requirements reasoning. While most computing education research targets introductory programming and most LLM-for-SE research focuses on code generation, graduate students lack pedagogical support for the strategic, values-centered thinking that distinguishes responsible software engineers. **Our primary contribution is Continuous Requirements Auditing (CRA)**: a specifically (and narrowly) scoped, expert-calibrated practice. CRA is the mechanical detection of requirement/implementation inconsistencies, delivered as continuous contextual feedback, and evaluated against learning measures independent of the system's own scoring.

TETHER's feedback to students draws on an integration of LLMs with Rhetorical Interface Theory. By operationalizing Norman's Signifier Theory [28], Bardzell's Rhetorical Framing [29, 30], and Fogg's Behavior Model [31], with Co-PI Rieder's expertise in digital rhetoric, TETHER transforms AI feedback from passive assessment into a pedagogically effective intervention. Our controlled experiments will yield evidence-based design principles for educational AI feedback applicable beyond TETHER.

Research outcomes include: (1) a validated framework connecting rhetorical design to measurable RE learning outcomes; (2) empirical evidence from controlled trials (n>600) on intervention design effects, including transfer to unscaffolded contexts; (3) understanding of effectiveness variation across student populations; (4) longitudinal evidence on persistence and professional transfer; and (5) open artifacts including assessment instruments, the calibrated TETHER system, and expert-annotated "Reference Set" requirements, enabling replication and extension.

6 Broader Impacts

Workforce Development. As generative AI automates routine coding, economic competitiveness depends on engineers who can govern AI-assisted development, ensuring systems align with stakeholder values and societal needs. This project cultivates strategic reasoning, values-centered design, and critical evaluation of AI solutions: the skills that industry identifies as essential. Open release of all materials enables broad adoption and industry partnerships extend impact to early-career professionals.

STEM Education Infrastructure. Traditional RE pedagogy with upfront documentation disconnected from practice fails to prepare students for continuous requirements reasoning. Our research embeds RE learning throughout development, producing durable infrastructure: validated assessment instruments, evidence-based AI feedback design principles, and open curriculum resources. Dissemination through SIGCSE and ICSE will reach faculty nationwide. By demonstrating how AI can scaffold professional reasoning without replacing judgment, this work establishes a replicable model for AI-supported education applicable beyond SE to any domain requiring navigation of competing values.

7 Results from Prior NSF Support for the PIs

PI Tim Menzies (NC State, Computer Science) is an ACM/IEEE Fellow with expertise in software analytics [33–38], three books on the topic [39–41], and recent SOTA work in active learning [42–44]. He is the editor-in-chief of the Automated Software Engineering journal and has earned $19M+ in competitive grants and graduated 18 Ph.D. students.

Recent Related Award: CCF-1302216, "SHF: Medium: Collaborative: Transfer Learning in SE," $271,553, 2013–2017.

Intellectual Merit: Developed novel methods for privacy-preserving data sharing in software engineering, enabling cross-project defect prediction without exposing proprietary source code. The LACE2 algorithm demonstrated that carefully obfuscated data retains sufficient signal for effective model training.

Broader Impacts: Enabled open science practices where organizations can share obfuscated datasets to build effective predictive models, applicable beyond SE to any data-intensive field requiring privacy protection. Graduated two Ph.D. students; released two public repositories.

Publications: [33–38]

Products: LACE2 toolkit and anonymized defect datasets available at Github at *github.com/ai-se*.

Co-PI David Rieder (NC State, English/CRDM) is an expert in digital rhetoric, composition studies, and how AI and algorithmic systems influence discourse and communication practices [27, 45, 46]. His work examines how computational tools reshape writing practices and pedagogical approaches in technical communication. *No prior NSF support.*

Senior Personnel Sarah Heckman (NC State, Computer Science) is a Teaching Professor, Director of Undergraduate Programs, and member of NC State's Academy of Outstanding Teachers. Her research spans help-seeking [47–49], automated grading [50], and teaming [51]. She chaired SIGCSE 2019–2020 and chairs the SIGCSE TS Steering Committee.

Most Relevant Award: DUE-1525173, "Collaborative: Transforming CSEd Research Through Appropriate Empirical Methods," $406,557, 2015–2022.

Intellectual Merit: Created the DEERS (Designing Empirical Education Research Studies) workshop series, training 70 computing education researchers across seven cohorts in rigorous empirical methods including experimental design, statistical analysis, and validity threats specific to CER contexts.

Broader Impacts: Trained seven cohorts of computing education researchers drawn from a wide range of institution types, including teaching-focused and resource-constrained institutions, producing 40+ peer-reviewed publications. Workshop materials freely available online, continuing to support new CER researchers.

Publications: [52–56]

Products: Project website with workshop materials [57].

References

[1] Marian Daun, Alicia M Grubb, Viktoria Stenkova, and Bastian Tenbergen. A systematic literature review of requirements engineering education. *Journal of Systems and Software*, 194:111456, 2022.

[2] B. W. Boehm. A spiral model of software development and enhancement. *IEEE Computer*, 21(5):61–72, May 1988. Risk-driven software process model.

[3] Haowei Cheng, Jati H. Husen, Yijun Lu, Teeradaj Racharak, Nobukazu Yoshioka, Naoyasu Ubayashi, and Hironori Washizaki. Generative ai for requirements engineering: A systematic literature review. *Software: Practice and Experience*, n/a(n/a).

[4] Ian Sommerville. *Software Engineering*. Pearson / Addison-Wesley, Boston, 9th edition, 2011. International Computer Science Series.

[5] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(8):1–79, 2024.

[6] Cigdem Sengul, Rumyana Neykova, and Giuseppe Destefanis. Software engineering education in the era of conversational ai: current trends and future directions. *Frontiers in Artificial Intelligence*, 7, 2024.

[7] Yishu Li, Jacky Keung, and Xiaoxue Ma. Integrating generative ai in software engineering education: Practical strategies. In *2024 International Symposium on Educational Technology (ISET)*, pages 49–53, 2024.

[8] Brittany Johnson and Tim Menzies. AI in State Courts: Navigating Innovation and Ethics With Andrea Miller . *IEEE Software*, 42(04):89–92, July 2025.

[9] Axel van Lamsweerde. Goal-oriented requirements engineering: A guided tour. *Proceedings of the Fifth IEEE International Symposium on Requirements Engineering*, pages 249–262, 2001.

[10] Mona Rahimi, Mehdi Mirakhorli, and Jane Cleland-Huang. Automated extraction and visualization of quality concerns from requirements specifications. In *2014 IEEE 22nd International Requirements Engineering Conference (RE)*, pages 253–262, 2014.

[11] Brittany Johnson, Yoonki Song, Emerson Murphy-Hill, and Robert Bowdidge. Why don't software developers use static analysis tools to find bugs? In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, pages 672–681. IEEE, 2013.

[12] Rita Garcia, Andrew Csizmadia, Janice L. Pearce, Bedour Alshaigy, Olga Glebova, Brian Harrington, Konstantinos Liaskos, Stephanie J. Lunn, Bonnie Mackellar, Usman Nasir, Raymond Pettit, Sandra Schulz, Craig Stewart, and Angela Zavaleta Bernuy. An international examination of non-technical skills and professional dispositions in computing – identifying the present day academia-industry gap. In *2024 Working Group Reports on Innovation and Technology in Computer Science Education*, ITiCSE 2024, page 124–174, New York, NY, USA, 2025. Association for Computing Machinery.

[13] Maryam Khan, Muhammad Azeem Akbar, and Jussi Kasurinen. Integrating llms in software engineering education: Motivators, demotivators, and a roadmap towards a framework for finnish higher education institutes. In *Proceedings of the 2025 29th International Conference on Evaluation and Assessment in Software Engineering Companion*, EASE Companion '25, page 182–191, New York, NY, USA, 2025. Association for Computing Machinery.

[14] James Prather, Juho Leinonen, Natalie Kiesler, Jamie Gorson Benario, Sam Lau, Stephen MacNeil, Narges Norouzi, Simone Opel, Vee Pettit, Leo Porter, Brent N. Reeves, Jaromir Savelka, IV Smith, David H., Sven Strickroth, and Daniel Zingaro. Beyond the hype: A comprehensive review of current trends in generative ai research, teaching practices, and tools. In *2024 Working Group Reports on Innovation and Technology in Computer Science Education*, ITiCSE 2024, page 300–338, New York, NY, USA, 2025. Association for Computing Machinery.

[15] Arif Nurwidyantoro, Mojtaba Shahin, Michel Chaudron, Waqar Hussain, Harsha Perera, Rifat Ara Shams, and Jon Whittle. Integrating human values in software development using a human values dashboard. *Empirical Software Engineering*, 28(3):67, 2023.

[16] Alexander Osterwalder and Yves Pigneur. *Business Model Generation: A Handbook for Visionaries, Game Changers, and Challengers*. Wiley, 2010.

[17] John Kania and Mark Kramer. Collective impact. *Stanford Social Innovation Review*, 9(1):36–41, 2011. Winter 2011.

[18] Tyler J. VanderWeele. On the promotion of human flourishing. *Proceedings of the National Academy of Sciences*, 114(31):8148–8156, 2017.

[19] B. S. Bloom, M. D. Engelhart, E. J. Furst, W. H. Hill, and D. R. Krathwohl. *Taxonomy of educational objectives: The classification of educational goals. Handbook I: Cognitive domain*. David McKay, 1956.

[20] L. W. Anderson and D. R. Krathwohl. *A taxonomy for learning, teaching, and assessing: A revision of Bloom's Taxonomy of Educational Objectives*. Longman, 2001.

[21] HIMSS Analytics. Electronic medical record adoption model (emram). Healthcare Information and Management Systems Society, 2005. Standard for 10,000+ healthcare facilities.

[22] Pravir Chandra and OWASP Foundation. *OWASP Software Assurance Maturity Model (SAMM)*, 2.0 edition, 2009. Industry standard for secure software lifecycles.

[23] Green Software Foundation. Green software maturity matrix. Linux Foundation, 2022. Framework for assessing green software practices.

[24] Microsoft. *Azure Well-Architected Framework: Reliability Pillar*, 2023. Standard for distributed cloud system resilience.

[25] Sabina-Cristiana Necula, Florin Dumitriu, and Valerică Greavu-Șerban. A systematic literature review on using natural language processing in software requirements engineering. *Electronics*, 13(11), 2024.

[26] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Timo Schick, Mavish Dwivedi, Abhishek Ring, Marta Neumann, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33:9459–9474, 2020.

[27] David M. Rieder. *Suasive Iterations: Rhetoric, Writing, and Physical Computing*. New Media Theory Series. Parlor Press, 2017.

[28] Don Norman. *The design of everyday things: Revised and expanded edition*. Basic Books (AZ), 2013.

[29] Jeffrey Bardzell and Shaowen Bardzell. Interaction criticism: An introduction to the practice. *Interacting with Computers*, 20(1).

[30] Jeffrey Bardzell. Interaction criticism and aesthetics. In *Proceedings of CHI '09*, pages 2357–2366. ACM, 2009.

[31] B. J. Fogg. A behavior model for persuasive design. In *Proceedings of the 4th International Conference on Persuasive Technology*, pages 1–7, 2009.

[32] James A. Kulik and Chen-Lin C. Kulik. Timing of feedback and verbal learning. *Review of Educational Research*, 58(1):79–97, 1988.

[33] Fayola Peters, Tim Menzies, and Lucas Layman. Lace2: Better privacy-preserving data sharing for cross project defect prediction. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, volume 1, pages 801–811. IEEE, 2015.

[34] Zhimin He, Fayola Peters, Tim Menzies, and Ye Yang. Learning from open-source projects: An empirical study on defect prediction. In *Empirical Software Engineering and Measurement, 2013 ACM/IEEE International Symposium on*, pages 45–54. IEEE, 2013.

[35] Tim Menzies, William Nichols, Forrest Shull, and Lucas Layman. Are delayed issues harder to resolve? revisiting cost-to-fix of defects throughout the lifecycle. *Empirical Software Engineering*, 22(4):1903–1935, 2017.

[36] Wei Fu, Tim Menzies, and Xipeng Shen. Tuning for software analytics: Is it really necessary? *IST*, 76:135–146, 2016.

[37] Rahul Krishna and Tim Menzies. Learning effective changes for software projects. *arXiv preprint arXiv:1708.05442. Under review; IEEE TSE*, 2017.

[38] R. Krishna, V. Nair, P. Jamshidi, and T. Menzies. Whence to learn? transferring knowledge in configurable systems using beetle. *TSE*, 2020.

[39] Tim Menzies, Laurie Williams, and Thomas Zimmermann. *Perspectives on Data Science for Software Engineering*. Morgan Kaufmann, Boston, 2016.

[40] Tim Menzies, Ekrem Kocagüneli, Leandro Minku, Fayola Peters, and Burak Turhan. Data science for software engineering: Sharing data and models. 2013.

[41] Christian Bird, Tim Menzies, and Thomas Zimmermann. *The art and science of analyzing software data*. Elsevier, 2015.

[42] Leandro Lustosa and Tim Menzies. Active learning and explainability for data-driven software engineering. In *Proceedings of the 43rd International Conference on Software Engineering (ICSE)*, 2021.

[43] Andre Lustosa and Tim Menzies. isneak: Partial ordering as heuristics for model-based reasoning in software engineering. *IEEE Access*, 2024.

[44] Tim Menzies et al. Xai for software engineering: Toward human-centered ai. *IEEE Software*, 38(4):30–38, 2021.

[45] Byron Hawk, David M. Rieder, and Ollie Oviedo, editors. *Small Tech: The Culture of Digital Tools*. Number 22 in Electronic Mediations Series. University of Minnesota Press, 2008.

[46] David M. Rieder. Eversion, ecology, touch, and rain: A post-pc rhetoric. *Journal of INDECS: Interdisciplinary Description of Complex Systems*, 19(1):1–12, 2021. Special Issue on Innovative Methodologies in Arts and Sciences.

[47] Zhikai Gao, Sarah Heckman, and Collin Lynch. Who uses office hours? a comparison of in-person and virtual office hours utilization. In *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education V. 1*, SIGCSE 2022, page 300–306, New York, NY, USA, 2022. Association for Computing Machinery.

[48] Matthew Zahn, Isabella Gransbury, Sarah Heckman, and Lina Battestilli. Assessment of self-identified learning struggles in cs2 programming assignments. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1*, ITiCSE 2023, page 264–270, New York, NY, USA, 2023. Association for Computing Machinery.

[49] Shao-Heng Ko, Matthew Zahn, Kristin Stephens-Martinez, Yesenia Velasco, Lina Battestilli, and Sarah Heckman. Relationships between computing students' characteristics, help-seeking approaches, and help-seeking behavior in introductory courses and beyond. In *Proceedings of the 2025 ACM Conference on International Computing Education Research V.1*, ICER '25, page 313–326, New York, NY, USA, 2025. Association for Computing Machinery.

[50] Sarah Heckman and Jason King. Developing software engineering skills using real tools for automated grading. In *Proceedings of the 49th ACM Technical Symposium on Computer Science Education*, SIGCSE '18, page 794–799, New York, NY, USA, 2018. Association for Computing Machinery.

[51] Kai Presler-Marshall, Sarah Heckman, and Kathryn T. Stolee. Improving grading outcomes in software engineering projects through automated contributions summaries. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*, pages 259–270, 2023.

[52] Ahmed Al-Zubidy, Jeffrey C. Carver, Sarah Heckman, and Mark Sherriff. A (updated) review of empiricism at the sigcse technical symposium. In *Proceedings of the 47th ACM Technical Symposium on Computing Science Education*, SIGCSE '16, page 120–125, New York, NY, USA, 2016. Association for Computing Machinery.

[53] Jeffrey C. Carver, Sarah Heckman, and Mark Sherriff. Training computing educators to become computing education researchers. In *SIGCSE '22: Proceedings of the 53rd ACM Technical Symposium on Computer Science Education*, SIGCSE'22, 2022.

[54] Sarah Heckman, Jeffrey C. Carver, Mark Sherriff, and Ahmed Al-Zubidy. A systematic literature review of empiricism and norms of reporting in computing education research literature. *ACM Transactions on Computing Education*, 22:1–46, 2021.

[55] Sarah Heckman, Jeffrey C. Carver, and Mark Sherriff. Designing empirical education research studies (deers): Creating an answerable research question (abstract only). In *Proceedings of the 2017 ACM SIGCSE Technical Symposium on Computer Science Education*, SIGCSE '17, page 737, New York, NY, USA, 2017. Association for Computing Machinery.

[56] Jeffrey C. Carver, Sarah Heckman, and Mark Sherriff. Designing empirical education research studies (deers): Creating an answerable research question (abstract only). In *Proceedings of the 49th ACM Technical Symposium on Computer Science Education*, SIGCSE '18, page 1051, New York, NY, USA, 2018. Association for Computing Machinery.

[57] Mark Sherriff, Sarah Heckman, and Jeffrey C. Carver. Designing empirical education research studies (deers).